

PORTADA

Calidad de Software

Semana 8 — Técnicas de prueba: caja negra, caja blanca, niveles y la pirámide de Cohn

Universidad de Antioquia · Ingeniería de Sistemas



REPASO RELÁMPAGO

Lo que ya tenemos de la semana 7

- Existen siete **tipos** de prueba: funcional, regresión, humo, rendimiento, seguridad, usabilidad, accesibilidad.
- No todos aplican siempre — descartar uno con justificación es válido.
- Humo es amplio y superficial; regresión es angosta y profunda.



TRANSICIÓN

Hoy: el "cómo" del diseño de casos

Ya sabemos **qué** tipo de prueba responde cada pregunta (semana 7) y **cuándo** se ejecuta cada una según el nivel del modelo en V (semana 2). Falta la pieza que cierra el círculo: **cómo** se decide, de forma sistemática y no al ojo, cuáles casos concretos vale la pena escribir.



CONCEPTO

Qué es una técnica de prueba

Es un **método sistemático** para derivar casos de prueba a partir de un artefacto — un requisito, una historia de usuario, o el código mismo — en vez de inventarlos por intuición.

Las técnicas se agrupan en dos grandes familias, según **qué información usan** para diseñar los casos: si miran la especificación desde afuera, o el código desde adentro.



CONCEPTO

Caja negra vs. caja blanca

Caja negra	Diseña casos a partir de la especificación — requisitos, historias de usuario — sin mirar el código. El sistema es una caja cerrada: importa la entrada y la salida.
Caja blanca	Diseña casos a partir de la estructura interna del código — sus sentencias, decisiones y caminos posibles.

Un buen plan de pruebas usa las dos — se complementan, no compiten.

TRANSICIÓN

Empezamos por caja negra

Cuatro técnicas clásicas de caja negra, de la más común a la más específica: particiones de equivalencia, valores límite, tablas de decisión y transición de estados.



CONCEPTO

Particiones de equivalencia

Divide las entradas posibles de un campo en **grupos donde, si un valor del grupo falla, se espera que todos fallen igual** — y viceversa. En vez de probar cada valor posible, se prueba **uno representativo por grupo**.

Ejemplo: un campo "edad" que solo acepta 18 a 65 tiene tres particiones: menor a 18 (inválida), 18–65 (válida), mayor a 65 (inválida). Probar 10, 30 y 80 cubre las tres — probar además 15 o 50 no añade información nueva.



CONCEPTO

Valores límite

Los defectos rara vez viven en la mitad de una partición — viven en sus **bordes**. Un error de "mayor que" en vez de "mayor o igual que" solo se ve probando justo en el límite.

Mismo ejemplo del campo "edad" (18–65): los valores a probar son 17, 18 y 19 (borde inferior) y 65, 66 (borde superior) — no el 30 del centro de la partición.

En la práctica casi siempre se combinan: particiones para saber dónde mirar, valores límite para saber exactamente qué probar ahí.



EJEMPLO

Partición + valores límite, combinados

Campo: "cantidad", acepta enteros de 1 a 10.

Caso	Valor	Partición	¿Por qué se incluye?
1	0	Inválida (baja)	Límite inferior inválido
2	1	Válida	Límite inferior válido
3	5	Válida	Representante del centro
4	10	Válida	Límite superior válido
5	11	Inválida (alta)	Límite superior inválido

CONCEPTO

Tablas de decisión

Sirve cuando el comportamiento depende de la **combinación de varias condiciones**, no de un solo campo. La tabla enumera cada combinación posible y la acción que le corresponde — nada queda implícito.

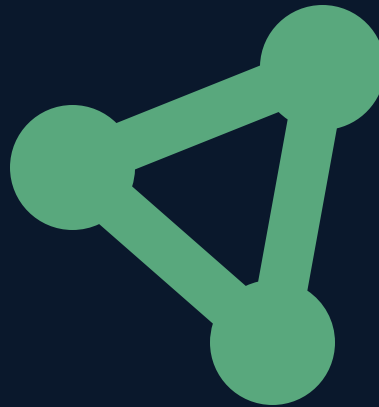
¿Usuario premium?	¿Monto > \$100?	Descuento aplicado
Sí	Sí	20 %
Sí	No	10 %
No	Sí	5 %
No	No	0 %

CONCEPTO

Transición de estados

Sirve cuando el sistema se comporta distinto según **en qué estado está**, no solo según la entrada actual. Se diseñan casos para cada transición válida — y, con la misma importancia, para las **transiciones inválidas** que deberían quedar bloqueadas.

Ejemplo: un pedido pasa por Pendiente → Pagado → Enviado → Entregado. Un caso válido prueba Pagado → Enviado. Un caso igual de valioso prueba que **no** se puede pasar de Pendiente directo a Entregado, saltándose los estados intermedios.



TRANSICIÓN

Ahora caja blanca

Las técnicas de caja negra ignoran el código a propósito. Las de caja blanca hacen justo lo contrario: usan la estructura del código para decidir qué tan a fondo se probó.



CONCEPTO

Qué es la cobertura de código

Es una **métrica**: qué porcentaje del código se ejecutó al menos una vez durante las pruebas. SonarCloud (semana 5) ya la reportaba — hoy se ve qué significa exactamente ese número, porque no todas las coberturas miden lo mismo.

Advertencia que se repite en todo el curso: alta cobertura no es sinónimo de buenas pruebas — solo dice qué se ejecutó, no si se verificó el resultado correcto.



CONCEPTO

Cobertura de sentencia

Mide qué porcentaje de **líneas de código** se ejecutó al menos una vez. Es la más fácil de alcanzar y la más débil: no garantiza que se haya probado cada decisión.

```
if edad >= 18:
    permitir_acceso()
else:
    denegar_acceso()
```

Un solo caso con `edad = 20` ejecuta la primera rama pero **no** la segunda — cobertura de sentencia parcial, aunque "la función se ejecutó".

CONCEPTO

Cobertura de decisión

Mide qué porcentaje de **decisiones** (los `if` , los `while`) tomaron tanto el valor verdadero como el falso al menos una vez. Es más exigente que sentencia: para el ejemplo anterior, hacen falta **dos** casos —uno con `edad >= 18` y otro con `edad < 18` — para cubrir el 100 %.

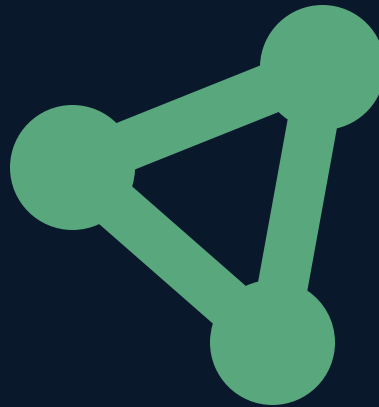
100 % de cobertura de decisión implica 100 % de cobertura de sentencia — lo contrario no es cierto.

CONCEPTO

Cobertura de camino

Mide qué porcentaje de las **combinaciones posibles de decisiones** (los "caminos" que puede recorrer la ejecución de principio a fin) quedaron cubiertas. Es la más exigente de las tres — y la que casi nunca llega al 100 %: el número de caminos crece exponencialmente con cada decisión nueva.

Por eso en la práctica se usa más como guía para encontrar combinaciones olvidadas que como meta numérica estricta.



CONCEPTO · SÍNTESIS

Las tres coberturas, de menos a más exigente

Cobertura	Qué exige	Exigencia
Sentencia	Cada línea se ejecuta al menos una vez	Baja
Decisión	Cada condición toma verdadero y falso	Media
Camino	Cada combinación de decisiones se recorre	Alta

Ninguna de las tres, ni siquiera al 100 %, garantiza ausencia de defectos — solo acota qué tanto del código quedó sin ejercitar.

TRANSICIÓN

Cuánto de cada técnica, en qué nivel

Con todas las técnicas sobre la mesa, falta la pregunta de distribución: si hay tiempo limitado, ¿dónde conviene invertir más esfuerzo — en pruebas unitarias, de integración, o de sistema completo?



CONCEPTO · ORIGEN

La pirámide de Cohn

Propuesta por **Mike Cohn** en su libro *"Succeeding with Agile"* (2009), organiza las pruebas automatizadas en tres capas según su costo y velocidad:

- **Base — Unitarias:** muchas, rápidas, baratas de mantener. Prueban una función o clase aislada.
- **Medio — Servicio/Integración:** menos cantidad, prueban cómo interactúan varios componentes.
- **Punta — Interfaz/UI:** pocas, lentas y frágiles. Prueban el sistema completo desde afuera.



CONCEPTO EN DISPUTA

¿La pirámide sigue vigente?

La pirámide asume que las pruebas de interfaz son lentas y frágiles por naturaleza — cierto en 2009, menos cierto hoy con mejores herramientas de automatización (semana 13). Algunos autores, como Kent C. Dodds, proponen un "**trofeo de pruebas**": menos pruebas unitarias aisladas, más pruebas de integración, porque estas últimas dan más confianza por caso escrito.

El principio que sí sobrevive a cualquier forma —pirámide o trofeo— es el mismo: **preferir muchas pruebas rápidas y baratas sobre pocas lentas y costosas**, sin importar cómo se llame la figura.



INTERACCIÓN

Ejercicio rápido: particiones y límites

5 minutos: un campo "porcentaje de descuento" acepta enteros de 0 a 50.

1. ¿Cuántas particiones tiene (válidas e inválidas)?
2. ¿Qué valores límite probarían?

Este ejercicio es el calentamiento del taller de hoy.

SÍNTESIS

Lo que hay que llevarse de hoy

1. **Caja negra** parte de la especificación; **caja blanca**, del código — se complementan.
2. **Particiones de equivalencia** agrupan entradas; **valores límite** prueban justo los bordes de esos grupos.
3. **Tablas de decisión** y **transición de estados** cubren combinaciones de condiciones y comportamientos dependientes del estado.
4. Cobertura de **sentencia, decisión y camino** exigen cada vez más — ninguna garantiza ausencia de defectos.
5. La **pirámide de Cohn**: muchas pruebas rápidas en la base, pocas lentas en la punta.



INSTRUCCIONES

Taller de hoy: casos sobre automationexercise.com

2 horas, en sala de Zoom asignada — equipo fijo de Fábrica-Escuela

Sobre el **formulario de registro** de automationexercise.com, cada equipo debe:

- Diseñar casos de prueba usando **particiones de equivalencia** y **valores límite** sobre al menos dos campos del formulario.
- **Ejecutar** esos casos contra el sitio real.
- **Reportar las discrepancias** encontradas entre el comportamiento esperado y el observado.

La entrega cierra al terminar las dos horas. Recuerden: al enviar, cada integrante reporta en privado quién trabajó.



CIERRE

Para la próxima semana

Antes de la clase de la semana 9, hay que llegar con esto leído:

- Introducción al proceso de pruebas. Ciclo de vida del defecto. Severidad vs. prioridad. Reporte de defectos.

Arranca la **Unidad 3 — Implementación del proceso de aseguramiento**. Y algo importante: **al inicio de la semana 9 se entrega el Trabajo 1** (25 % de la nota) — el plan de pruebas y diseño de casos aplicado al proyecto de Fábrica-Escuela, con todo lo visto hasta hoy.

